# HeavyHeader

C/C++ 构建效能治理平台

用编译器自己的输出，回答「这个头文件让我多等了几秒」

第四届开放原子大赛 · 2026 开源行业解决方案创新赛
赛道一 · 开源基础软件与解决方案

项目负责人：王子周
申报主体类型：开源社区（个人）
日期：2026 年 10 月 17 日

代码仓库：`https://github.com/wzzrambler/heavyheader`

Apache-2.0 · 本作品以开源方式提交

# 摘要

大型 C/C++ 工程的构建时间是真实的成本：一次全量构建动辄数十分钟到数小时，CI 集群机时与工程师等待时间都花在里面。构建慢的最大单一来源往往不是算法，而是头文件失控——一个被数万个编译单元传递包含的重量级头文件，会让整个项目反复为它付费。

现有工具分别解决了这个问题的一段，本文先把来源交代清楚：Header Hero（Bitsquid，2011）用「头文件行数 × 被包含次数」给出成本排序，本工具的归因行数与 Blowup Factor 公式即来自它；ClangBuildAnalyzer 用 `-ftime-trace` 给出逐头文件的累计毫秒与被包含次数；IncludeGuardian 给出七类结构建议；deheader（ESR，2010）注释掉一条 include 再编译来验证可删性。但它们都不回答「删掉这一条，这次构建具体省多少时间、多少内存、多少体积」——要么没有反事实，要么只有二值判断。HeavyHeader 补的正是这一段：让真实编译器报告自己做了什么，对候选做 A/B 两次真实编译，用实测差值给出 REMOVE / KEEP / HUMAN 三态结论。

项目已实现四个命令：`scan`（成本归因与报告）、`verify`（实测验证）、`gate`（CI 门禁）、`compare`（前后对比），支持 GCC 9+、Clang 17+ 与 MSVC 三种工具链，附带 447 个单元测试与完整开源治理文件。

除成本归因外，项目还做了三件现有工具不做的事：模板实例化成本归因（实测包含 nlohmann/json.hpp 的文件有 39% 构建时间花在模板实例化上）、预编译头收益估算、前向声明建议。

在 4 个真实开源项目（fmt / nlohmann-json / spdlog / cpp-httplib）上共扫描 54 个编译单元、1430 万行头文件解析量，产出 87 个成本候选。候选清单只含项目自己的头文件：系统头与第三方头照常计入解析量统计，但从不建议删除——「建议删掉 <vector>」这类无法执行的条目会把可操作项全部埋掉。在刻意构造的基准工程上，它准确识别出唯一一条未使用的 include 并实测节省 200 ms（38.5%）。

关键词：构建效能；头文件成本归因；include 治理；开发者工具；CI 门禁；编译器诊断输出

# 一、问题背景与需求依据

## 1.1 构建慢的钱花在哪

一个 C++ 项目的预处理与解析成本，一阶近似正比于「编译器被迫读了多少头文件文本」。每个被解析的头文件，在每个引用它的编译单元里贡献一次自己的行数。这意味着一个 4700 行的头文件被 3000 个编译单元引用时，项目要为它解析 1410 万行——而这还只是它自身，不含它传递拉进来的一切。

这个成本是隐形的。开发者看到的是「编译好慢」，看不到是哪几行 `#include` 造成的。现有的分析工具要么只做结构展示，要么只做正确性判断，没有一个把成本和具体代码行对应起来。

### 1.2 现有工具的空白

下表是本项目立项时对同类工具的逐项核读。对比的重点不是功能有无，而是「能否回答成本问题」——这是本项目的立足点。

| 工具 | 回答什么问题 | 不能回答什么 |
| --- | --- | --- |
| include-what-you-use | 基于 AST 判断 include 是否必需 | 不给成本量化，无法排序优先级 |
| cpp-dependencies | 分析模块间依赖关系 | 面向架构评审，不回答构建耗时 |
| include-gardener | 可视化 include 关系图 | 只展示结构，不做归因 |
| IDE 的 Include Graph | 查看单个文件的包含树 | 手动、单文件、无项目级视图 |
| HeavyHeader（本项目） | 这个头文件让项目多解析了多少行、多等了几秒，删掉它能省多少 | 精确的语义级未使用判定（见第九章） |

> 需要说明的是：上表对本项目一栏的描述是设计目标，其兑现程度由第四章的实测数据检验，不由声明检验。

### 1.3 目标场景

本项目锚定的场景是汽车电子、工业软件、通信设备厂商的 C++ 单体仓库（monorepo）。这些团队的共同特征：代码量在数十万到数百万行、编译单元数以千计、一次全量构建数小时、CI 集群机时是可观成本项。在这些团队里，「构建效能治理」是有明确投资回报的工程命题，而不是个人偏好。

首版不限定使用者必须来自上述行业——任何 C/C++ 项目都能用。但项目的问题定义、指标设计和验收标准都以上述场景的规模为前提，因为只有那个规模下，本章描述的成本才显著到值得治理。

## 二、核心方法论：不自己解析 C++

整个项目只有一条设计原则，其他一切都是它的推论：不自己解析 C++。工具向真实编译器提问，读取它的回答。

这不是懒，是正确性要求。一个手写预处理器在遇到模板元编程、条件编译、宏拼接时会静默给出错误答案，而这种错误比崩溃危险得多——崩溃会被发现，错误答案不会。让编译器自己报告它做了什么，分析结果就在构造上是精确的。

## 2.1 一次编译，三个问题

| 问题 | 怎么问编译器 |
| --- | --- |
| 你包含了什么，嵌套多深? | GCC/Clang <font face='Cascadia'>-H</font>，MSVC <font face='Cascadia'>/showIncludes</font> |
| 时间花在哪? | GCC/Clang <font face='Cascadia'>-ftime-report</font>，MSVC 墙钟计时 |
| 哪些头文件缺 include guard? | <font face='Cascadia'>-H</font> 输出的 trailer |

三个问题在同一次编译里全部得到回答——`-H` 和 `-ftime-report` 都写 stderr。分成两次调用会让扫描耗时翻倍，更糟的是两次之间头文件若发生变化，两边数据会互相矛盾。

## 2.2 归因行数：成本的度量

```
attributed_lines(h) = line_count(h) × tu_reach(h)

tu_reach(h) = □□□□□□□□ h □□□□□□□□
```

这个乘积衡量「整个项目为 h 付出了多少行解析量」。它是对同一份代码确定的，不随机器负载波动——这一点很重要，因为它是 CI 门禁唯一可依据的指标。

## 2.3 独占闭包：收益的计算

删掉一条 include 后，唯一会停止被解析的头文件集合叫它的独占闭包：

```
exclusive(d) = closure(d) − ( □ closure(d′) )
d′ □ direct_includes(t), d′ ≠ d
```

这是本项目与普通 lint 工具的本质区别。一个头文件如果能从别的 include 到达，那么删掉当前这条 include 并不会让它停止被解析——钱已经花了。只有独占闭包里的头文件才是删掉这条 include 的真实收益。不做这个计算的工具，会把大量零收益的建议排在真收益前面。

## 2.4 重复解析检测：确定性的必赚项

`-H` 只为实际处理过的包含打印一行。带 `#pragma once` 或 guard 宏的头文件，第二次及以后的包含会被跳过，因此无论多少个头文件包含它，`-H` 里只出现一次。于是「同一编译单元内出现多次」在物理上只可能是缺少 guard。

修复（补 guard）永远安全，且收益立竿见影。这是报告里唯一标为「必赚项」的分类——不需要实测，因为逻辑上不可能出错。

# 三、完整任务示例（可手算复核）

本章用一个 45 行的构造工程走完「扫描 → 定位 → 验证」全流程。所有数字都可手工复核，工程在仓库 benchmarks/cases/demo。

## 3.1 工程构造

```
benchmarks/cases/demo/
├── include/
│   ├── core.h # □□□ TU □□□□□□□□□
│   ├── heavy.h # □□□□□□□
│   ├── bloated.h # □□ lonely.h □□□□□□□□□
│   ├── lonely.h # □ main.cpp □□□□□□□□
│   ├── unguarded.h # □□□□ guard
│   └── mid.h
└── app/
    ├── main.cpp # □□ lonely.h□□□□□□□□□
    └── util.cpp
```

## 3.2 扫描结果

```
$ heavyheader scan benchmarks/cases/demo \
--compiler g++ --std c++17 -I benchmarks/cases/demo/include

2 TU · 425 □□□□ · 206,106 □□□□□□□□ · 5 □□□□□□□□□ 1 □□□□□□□□
```

成本热点表把 core.h 排在首位（被两个编译单元引用），而 lonely.h 被标记为「疑似未使用」并因独占闭包最大（22,312 行）排在候选首位。缺 guard 的 unguarded.h 出现在「必赚项」表中：解析 3 次、涉及 2 个 TU。

## 3.3 实测验证

成本模型再严谨也仍是模型。verify 命令注释掉候选 include、重新编译、计时对比，每侧重复 3 次取最小值（噪声只会让测量变慢，不会变快，所以最小值是最接近真实成本的估计）：

```
$ heavyheader verify benchmarks/cases/demo --top 3 --runs 3

## lonely.h ← main.cpp
□□□improved — □□ 520 ms → □□□ 320 ms
□□□□□200 ms (38.5%)

## core.h ← util.cpp
□□□compile-error — □□□□□□□□□
util.cpp:3:14: error: 'Core' was not declared in this scope

## core.h ← main.cpp
□□□compile-error — □□□□□□□□□
main.cpp:6:14: error: 'Core' was not declared in this scope

□□□1/3 □□□□□□□□□
```

注意最后那个「1/3」。工具没有把三个候选都说成可删——它把两个正在使用的头文件正确判为「删了会编译失败」，只对真正未使用的那个给出收益数字。这个区分是第四章在成熟项目上验证的重点。

### 3.4 明确结论：每个候选一个「值不值得删」

「疑似未使用」是可能性，不是判断。决策层把探针证据收敛成三个明确结论之一：建议删除（编译通过且耗时下降超过 2% 噪声线）、保留（编译器已否决，不必再试）、需人工判断（编译可行但收益在噪声内，附倾向与理由）。未实测的候选显示「未实测」——没测过就不下判断。

每个结论按三个维度列依据：功能影响（还能不能编、哪些标识符消失）、维护成本（整个项目少解析多少代码、还有几个使用单元）、构建体验（编译时间、编译器内存分配、目标文件体积各差多少）。工具链不提供的维度写「该工具链不提供」，不写成 0；报告附完整复现命令。

在 nlohmann/json 测试目标上（222 个候选全部真实编译）：建议删除 1、保留 187、需人工判断 34。唯一一条建议删除的三个维度全部实测——去掉 windows.h 后编译 1610 ms → 910 ms（−43.5%）、GGC 内存 301,052 kB → 168,960 kB、目标文件 63,004 B → 62,972 B（−32 B，收益全部在编译时间侧）。187 个「保留」与 1 个「建议删除」同样是一等公民：把试过、不行记录下来，和把有效改动找出来一样重要。

### 3.5 四道门槛：我们把自己的卖点证伪了一次

在写这份文档之前，项目按预注册的判据跑了四道门槛，结果并不全是好消息。把它写在正文而不是附录里，是因为评审有权先看到不利数据。

G1 公平对照（部分通过）：与 ClangBuildAnalyzer 在同一批 25 个编译单元、同一编译器、同一候选规则下对照。把两边口径统一成「自身成本 × 次数」后，Top-10 重合 7/10、Spearman ρ ≈ 0.68——只能说明中度一致，不能说明我们更准。

G2 优化闭环（成立，但有限）：第一版协议在 nlohmann/json 上跑出 0/10，当时据此判为不成立。把那个 0 拆开看，它有三个来源，逐个修掉之后结论变了。① 候选空间太窄——只看编译单元自己的 include，83 条候选里只有 3 条疑似未使用；把 include 图摊到所有文件后有 432 对，一条多余 include 的收益是「独占行数 × 受影响的编译单元数」。② 测量协议太钝——先测完基线再测补丁，机器漂移会整块算成收益，聚合 A/A 实测 2.005%，改成交替测量后降到 0.644%。③ 一处判定被自己污染——「#include <regex>」这一行的文件名会进入包含者的标识符集，用法判定于是恒判「在用」。修完之后：doctest（24 单元）上带可行性闸门的5 条推荐里 1 条被证实可删且真省时——删掉一个公共头里的 `#include <unordered_set>`，24 个单元全部编译通过，聚合耗时 20556 → 19937 ms（−3.01%）并通过复测；关掉闸门做消融对照是 0/5。换到 json（25 单元）是 0/5，且闸门一条都不推荐——20% 是 doctest 的数字，不是工具的数字。另做注入-召回：人工注入一条确实多余且昂贵的 include，头文件级排 1/17、单元级排 25/61，两条都被实测证实（−6.82%/−6.14% 与 −5.77%/−6.65%，各跑两遍独立

运行都过线）。因此撤回「自动找到优化」的说法，但可以说在真实工程上产出过一条经复测的优化。

G3 门禁（成立，但范围有限）：注入一次已知回归后 3/3 检出；无回归时 3/3 通过、零误报；构建损坏导致编译单元减少时 3/3 拦截（这一条是实测中发现缺陷后修的：旧逻辑会因为总量「下降」而放行）。已知盲区：行数不变但编译变慢的回归测不出来。

G4 工程开销（成立）：同任务下扫描开销为纯编译时间的 1.15×（GCC）/ 1.20×（Clang），峰值内存 9.2 MB，三次运行结果完全一致。

把四条放在一起，产品的诚实定位是：它不承诺替你找到优化，它承诺把「看起来能省」变成「确认能省」或「确认不能省」，并把这个结论固化成能拦住回归的基线——并且在 doctest 这个真实工程上，它确实产出过一条经复测的优化；命中率是 1/5，换一个工程会变成 0/5，这两个数字都写在上面。这个定位比原来的说法窄，但每一条都有可复现的实验数据支撑。

# 四、真实项目验证

第三章的数字来自刻意构造的工程，只能证明工具能跑。本章用真实开源项目检验它是否可信——尤其是它会不会误报。

## 4.1 在 fmt 上的扫描

对 4 个真实开源项目扫描。每个项目的 flags 都是实际跑通过、能产出干净数据的那一组，写在仓库 `benchmarks/run_all.py` 里而非让人记——复现不了的数字在评审眼里等于没测。

| 项目 | 编译单元 | 依赖边 | 头文件解析量 | 成本候选 | 疑似未使用 |
|---|---|---|---|---|---|
| fmt | 3 / 1 失败 | 651 | 331,016 | 4 | 0 |
| nlohmann-json | 25 / 0 | 9,294 | 4,866,676 | 24 | 0 |
| spdlog | 8 / 17 失败 | 3,246 | 1,579,902 | 41 | 0 |
| cpp-httplib | 18 / 7 失败 | 14,724 | 7,523,316 | 18 | 0 |
| 合计 | 54 成功 | 27,915 | 14,300,910 | 87 | 0 |

唯一失败的 `src/fmt.cc` 已确诊：该文件是 C++20 module（第 8 行 `module;`），MinGW GCC 16.2 不支持 `-fmodules`，换 `--std c++20` 同样报错。这是工具链限制，不是工具缺陷，且失败被如实报告而非静默跳过——报告顶部会写明「1 个编译单元失败，数字可能偏低」。

### 4.2 关键结果是那个 0

fmt 是维护良好的库，它的 include 确实是干净的。工具给出 6 个成本候选，对前 3 个逐一实测：

```
## windows.h ← os.cc
□□□compile-error — os.cc:90:9: error:
'FORMAT_MESSAGE_ALLOCATE_BUFFER' was not declared in this scope

## format-inl.h ← format.cc
□□□compile-error — format.cc:10:1: error:
'FMT_BEGIN_NAMESPACE' does not name a type

## os.h ← os.cc
□□□compile-error — os.cc:102:12: error:
'fmt' does not name a type
```

三个全部正确判定为「删了会编译失败」。一个只会喊「你可以删掉这个」的工具，第一次误报就会被人关掉。HeavyHeader
在成熟项目上报告「没有可删的」，同时仍然量化出成本集中在哪——这个对比本身就是价值。

把 4 个项目合起来看：87 个成本候选中 0 个疑似未使用。这个比例本身就是结论——成熟项目的 include 大多是干净的，工具没有为了凑数字而喊。反过来说，当它真的报出一个候选时（见第三章），那个候选值得认真对待。

与之对照，第三章构造的未使用 include 被准确识别并实测省 38.5%。该报的报，不该报的不报，两边都有数据。

### 4.3 这些数字的边界

如实说明：本章数字取自 GCC 16.2.0 (MinGW-w64) + Python 3.13，随编译器版本、平台、机器负载变化，不应逐位复现。可复现的是结论方向：候选数是否为 0、verify 是否报 compile-error。完整复现步骤见仓库 `benchmarks/results/README.md`。

# 五、现有工具覆盖不到的三件事

第一章已经交代清楚：成本排序、逐头文件耗时、结构建议、注释后重编，这四件事分别有成熟工具在做（Header Hero / ClangBuildAnalyzer / IncludeGuardian / deheader）。本章不重复它们，只讲三处现有工具覆盖不到的细节，以及每一处的证据边界——哪些是精确值，哪些只是启发式线索。

## 5.1 模板实例化成本归因

`-ftime-report` 从 GCC 9 起就报告 `template instantiation` 阶段，但对模板重型 C++ 它常是最大单项。需要说明的是：ClangBuildAnalyzer 已经在 Clang 上展示「最慢实例化的模板」（数据来自 `-ftime-trace`）。我们补的是另外两件事：把同一份归因带到 GCC 与 MSVC（靠 `-ftime-report`），以及把成本按头文件摆出来，而不只是按模板名列一个全局 Top-N。

实测：一个包含 `nlohmann/json.hpp` 的文件，模板实例化 `0.44s` / 总耗时 `1.13s`，即 39%。在 nlohmann-json 的 25 个编译单元上，全项目模板实例化占 42.8%，22 个单元占比超过 25%。

两部分，诚实程度不同，报告里分别标注：

| 指标 | 来源 | 性质 |
| --- | --- | --- |
| 每个 TU 的秒数与占比 | -ftime-report | 精确 |
| 每个头文件的「模板表面」 | 文本启发式（template< 声明数 × TU 数） | 线索，非结论 |

按头文件的归因编译器给不了——它只报告阶段总量。所以「模板表面」只能提示成本可能来自哪里。在 nlohmann-json 上，头号嫌疑是 `json.hpp`（173 个模板声明 × 24 个 TU）。系统头文件（如 `<type_traits>`）的模板声明数通常更高，但用户改不了，故不列入报告。

## 5.2 预编译头（PCH）收益估算

PCH 是慢构建里杠杆最大的单项改动，但多数团队靠猜决定放什么——通常是「大家都包含的头文件」，这经常是错的。本工具从 include 图算出该放什么。

```
□□ PCH □□□□□□□ P□
□□ = Σ lines(h), h □ closure(P) × □□□□□ P □ TU □
− Σ lines(h) □□□□□□□□□□

// nlohmann-json □□
□□□□ 5 □□□□□□□□□□ 139,180 □□
24/25 □□□□□□□□□□□□□□□□□□ 96%□□
□□□□□ 3,201,140 □□□□□□□□□□□□□ 65.8%□□
```

四条限制写进报告本身：行数是代理不是秒数；上限估计（真实 PCH 必须是每个 TU 的第一个 include 且字节一致）；未计 PCH 失效；本工具不生成 PCH。

### 5.3 前向声明建议

把 include 换成前向声明，是编译时间优化里收益最高的一类：编译器读一行文本，而不是整个头文件的传递闭包。也是最容易出误报的一类，因为「只以指针/引用形式使用」是语义属性，而我们在文本上判断。

所以规则刻意保守，且报告给出具体类型名、使用行号、阻断类型：

```
| □□□□ | include | □□□□□□□□ | □□□□ | □□□□ |
| example\hello.cc | httplib.h | Request | 14 | Response□Server |
```

「阻断类型」非空表示该头文件里还有其他类型需要完整定义，不能整个 include 换掉，只能把可前向声明的类型拆出去。开发过程中修掉一个真实的假阴性：`int read(Value* v) {{ return v->n; }}` 里 `Value` 只以指针位置出现，纯文本检查会判为可前向声明，但 `v->n` 需要完整类型。补上「跟踪指针变量是否被解引用」后正确判为阻断。

## 六、平台能力：从工具到流水线的一环

如果只能跑一次、出一份报告，它是个玩具。HeavyHeader 的四个命令覆盖「发现 → 验证 → 门禁 → 对比」的完整闭环：

| 命令 | 作用 | 在流水线中的位置 |
| --- | --- | --- |
| <font face='Cascadia'>scan</font> | 成本归因，产出 Markdown / HTML / JSON 报告，可存基线 | 本地分析、CI 报告 |
| <font face='Cascadia'>verify</font> | 注释掉候选 include、重编译、实测耗时差异 | 采纳建议前的确认 |
| <font face='Cascadia'>gate</font> | 成本相对基线回归超阈值则非零退出 | PR 门禁 |
| <font face='Cascadia'>compare</font> | 对比两份扫描，定位成本上升的具体头文件 | 版本间对比 |

### 6.1 CI 门禁为什么可信

门禁只对比确定性的归因行数和相对比例，不对比墙钟时间——后者随负载波动，拿它做判定会产生误触发。而一个误触发的门禁会被关掉，关掉的门禁保护不了任何东西。默认阈值 5% 是「明显变化」与「正常波动」之间的实用分界。

失败的构建会指名道姓：

```
□□□□□
- header cost regressed +17.1% (206100 -> 241300 attributed lines),
```

```
budget is +5.0%
worst offender: include/bloated.h 0 -> 22312 lines (+100.0%)
```

一个只说「成本回归 +12%」的门禁毫无用处；能定位到具体文件和具体 include 行的门禁才会被留下来。

### 6.2 报告是自包含单文件

HTML 报告不引 CDN、不引框架、图表是手写内联 SVG。这不是技术偏好，是需求：报告要贴进 PR、要发给别人、要在没网的机器上打开。一个在这些场景下显示白页的报告毫无价值。数据量级（几十个头文件、几十个历史点）也远不够撑起一个图表库的开销。

## 七、技术路线与架构

### 6.1 分层

```
src/heavyheader/
├── drivers/ □□□□□□□□GCC/Clang/MSVC □□□□□□
├── parse/ □□ -H □□□□-ftime-report □□□make □□□□□
├── attrib/ □□□□□□□□□□□□□□□□□□□□□□□□□□□□
├── verify.py □□□□□□□ include → □□□ → □□
├── scanner.py □□□□□□ TU → □□□□□□□□ → □□
├── baseline.py □□□□□□□CI □□
└── report/ Markdown / HTML / JSON □□
```

关键性质：`attrib/` 和 `parse/` 是纯函数，不碰编译器、不碰文件系统、不碰网络。核心逻辑可以被彻底单元测试——归因引擎的全部测试用例都不需要安装任何编译器。

### 6.2 为什么核心用 Python 而不是 C++

项目的使用者是 C++ 开发者，但工具本身用 Python 写。理由是工作性质：它的主体是调用子进程、解析文本、建图、排序、渲染报告——这是 I/O 密集型而非计算密集型，Python
的开发速度优势直接转化为交付速度，且零第三方运行时依赖意味着它能在最朴素的 CI 容器里跑起来。编译慢的 C++ 项目不该为了分析工具再拖一套运行时进去。

需要声明的是：这个选择有代价。Python 的解释器启动开销在每次调用编译器时都会被放大，而 libclang 绑定在 Python 生态里更成熟。Roadmap 中的「精确模式」会利用后一点。

### 6.3 支持的工具链

| 编译器 | 依赖树 | 分阶段计时 |
| --- | --- | --- |
| GCC 9+ | <font face='Cascadia'>-H</font> | <font face='Cascadia'>-ftime-report</font> |
| Clang 17+ | <font face='Cascadia'>-H</font> | <font face='Cascadia'>-ftime-report</font> |
| MSVC | <font face='Cascadia'>/showIncludes</font> | 墙钟计时（无等价文本格式） |

有 `compile_commands.json` 时严格按其中的 flags 编译——同一份代码在不同 flags 下的数字没有可比性。

## 八、诚实的边界：不做什么

这一章和前面几章同样重要。一个工具如果把局限藏起来，使用者就无法判断什么时候该信它、什么时候不该。

| 不做什么 | 为什么 | 怎么办 |
| --- | --- | --- |
| 不判定 include 是否「未使用」的语义真相 | 文本启发式只能回答「源码里有没有出现该头文件声明的标识符」 | 权威结论由 verify 给出：真的删掉、重新编译 |
| 不做精确的 AST 级分析 | 要精确判定需要 libclang，会把使用门槛抬到需装整套 LLVM 工具链 | Roadmap v0.3 作为可选 extra，默认路径保持零依赖 |
| 不改写代码 | 自动重构一个它不完全理解的代码库，风险远大于收益 | 工具给建议，人做决定；verify 只在临时目录做实验 |
| 不跨机器比较绝对耗时 | 墙钟时间随负载波动 | 门禁只对比确定性的归因行数与相对比例 |
| 不信任编译器的自报时序 | 实测发现 GCC on MinGW 会把某阶段报成 584 年（见 8.1） | 用进程墙钟做上界校验，不可能的的行丢弃并警告 |

### 7.1 一个真实的编译器缺陷

开发过程中在 fmt 上发现 `baseline_seconds` 报出约 1170 年。根因不在本项目：GCC 16.2.0 (MinGW-w64) 的 `-ftime-report` 在编译包含 `<windows.h>` 的文件时，会把 `phase parsing` 报成 `18446744070.35` 秒——即 2^64 纳秒折成秒，约 584 年。

解析器忠实解析了垃圾，于是报告里出现一个看起来权威实际荒谬的数字。修法：driver 本来就用 `perf_counter` 测量了进程墙钟时间，而一个编译阶段不可能比包含它的进程更长。以此为上界校验，超出的行丢弃，并 surfaced 成警告而非静默修正——用户有权知道他们的工具链有缺陷。

这件事本身说明了「不自己解析 C++」这条原则的代价：你把编译器当权威，就得防着它说谎。所以工具对编译器的每个输出都保留交叉校验的能力。

### 7.2 已知局限汇总

① 行数不等于解析难度，重型模板头的权重被低估；② 默认 `-fsyntax-only`，不含代码生成阶段成本；③ 使用状态是文本启发式，为副作用而包含的头文件（如 `<iostream>` 的流初始化）会被误判；④ 增量编译（ccache、distcc）会改变耗时分布，跑 verify 前需清缓存。完整列表见仓库 `docs/methodology.md` 第 9 节。

## 九、开源治理与社区

项目从第一行代码起就按可协作的标准组织，而不是跑出demo再补文件。大赛评分中「开源治理」占 20%，但我们做这些的理由更直接：一个没有 CONTRIBUTING 的项目收不到贡献，没有行为准则的社区留不住人。

| 文件 | 作用 |
| --- | --- |
| README.md | 问题定义、与同类工具对比、快速开始、诚实的边界 |
| LICENSE (Apache-2.0) | 对贡献者和使用者都明确，含专利授权条款 |
| CONTRIBUTING.md | 按优先级列出「什么贡献最被需要」，避免力气花错方向 |
| CODE_OF_CONDUCT.md | 基于 Contributor Covenant 1.4 |
| SECURITY.md | 威胁模型：编译器输出是不可信输入、临时文件、不联网 |
| ROADMAP.md | v0.1–v0.4 路线图 + 「明确不做的事」清单 |
| CHANGELOG.md | 语义化版本，记录每个修复的根因 |
| docs/methodology.md | 每个指标的定义、局限、以及「查不到就不该出现在报告里」 |
| docs/architecture.md | 分层设计与刻意选择的理由 |
| .github/ | Issue 模板（bug/feature）、PR 模板、CI 工作流 |

### 8.1 CI 与自举

CI 覆盖 Linux / Windows / macOS 三个操作系统 × Python 3.10–3.13 四个版本，外加一个 self-scan 任务：在 CI 里构造一个真实 C++ 工程并用 HeavyHeader 扫描它，断言归因行数非零、编译单元被发现。这一步同时也是集成测试——它验证工具在非 Windows、非 MinGW 的工具链下也能正确解析编译器输出。

### 8.2 为什么选 Apache-2.0

三个理由：对贡献者和使用者都明确；含专利授权条款，这对一个要进企业 CI 的工具不是形式问题；与生态中大量 C++ 项目的许可证兼容。

### 8.3 测试策略

447 个单元测试。样本优先用真实编译器输出逐字复制——例如 `tests/test_parsers.py` 里的 `-H` 和 `-ftime-report` 输出是从 GCC 16.2 真实运行复制的。这意味着换编译器版本时这些测试会最先报警。

## 十、开发过程与后续计划

### 9.1 已修的真实缺陷

开发过程中修掉 5 个 bug，每个都有回归测试。列出它们是因为它们解释了这个项目为什么是现在这样：

| 缺陷 | 症状 | 为什么难发现 |
| --- | --- | --- |
| 独占闭包恒为空 | 所有项目都报「无可优化项」 | 症状与「确实没问题」在输出上完全无法区分 |
| make 规则按第一个裸冒号切分 | Windows 盘符冒号被误判为分隔符，目标被截断 | 只在 Windows 路径上出现，Linux 开发不会遇到 |
| MSVC 缩进深度直接除步长 | 所有嵌套 include 都报成深度 0 | MSVC 最浅层本身有空格，不是从 0 开始 |
| usage_label 漏序列化 | 基线文件丢失使用状态信息 | 模型加了字段但序列化器没写，两边不同步 |
| GCC 坏计时器（见 8.1） | 构建成本报出 1170 年 | 编译器自己的输出看起来是权威的 |

### 9.2 Roadmap

| 版本 | 内容 | 状态 |
| --- | --- | --- |
| v0.1 | 可用的核心：三种工具链解析、成本归因、四个命令、72 测试 | 已发布 |
| v0.2 | 从工具到平台：历史存储与趋势服务、增量扫描、PR 评论机器人、多编译器对比 | 计划中 |
| v0.3 | 精确模式：libclang AST 后端（可选依赖）、前向声明建议、PCH 收益评估 | 计划中 |
| v0.4 | 生态集成：CMake、Bazel/Meson、VSCode 插件、与 IWYU 输出互转 | 计划中 |

Roadmap 里还有一份「明确不做的事」清单：不做通用静态分析器、不自动修改代码、不做云服务、不追求支持所有编译器。这份清单和功能列表同样重要——它防止项目范围蔓延。

### 9.3 关于团队

本项目由个人开发者以「开源社区」主体类型申报，独立完成需求、设计、实现、测试与文档。这是一个明确的事实，不是需要修饰的短板：项目从零到可运行的全部工作都在公开仓库的 git 历史里，每个 commit 都说明了「为什么」而不只是「做了什么」。

后续开发按 Roadmap 推进，欢迎社区参与。CONTRIBUTING.md 开头按优先级列出了「什么贡献最被需要」——排第一的是真实项目的扫描结果反馈，因为那是唯一能检验工具在真实代码库上是否给出荒谬结论的方式。

## 十一、参考资料

### 10.1 实际核读过的同类项目

以下项目在立项时逐项核读，用于第一章的对比表。核读日期为 2026 年 10 月。

```
R1 include-what-you-use — https://include-what-you-use.org/
R2 cpp-dependencies — https://github.com/tomtom-international/cpp-dependencies
R3 include-gardener — https://github.com/fedorbondarczuk/include-gardener
R4 fmtlib/fmt — https://github.com/fmtlib/fmt □□□□□□□□□□□
```

### 10.2 实际使用的编译器文档

```
R5 GCC □□□-H□-ftime-report□-M/-MM □□□□
R6 MSVC □□□/showIncludes □□□□□
R7 C++ □□□include guard □ #pragma once □□□
```

### 10.3 未引用但想说明的事

本项目没有引用政策文件或学术论文来支撑其必要性。这不是疏漏，是刻意的：我们没有找到一份能直接支撑「头文件成本归因」这个具体问题的权威文献，而为了显得严谨去引一份关系不大的政策条文，比不引更糟。

项目的必要性论证来自三件可验证的事：第一章的成本模型（可手算）、第三章的构造工程（可复现）、第四章的真实项目数据（可复现）。如果这三件事不成立，引多少文献也救不了它。

本作品以 Apache-2.0 许可证开源提交。代码仓库、测试、治理文件与基准数据均已随作品材料提交。

> 代码仓库：`https://github.com/wzzrambler/heavyheader`
> 演示视频：随作品材料提交（3-5 分钟完整片）
> 复现方式：见仓库 `benchmarks/README.md`，本文所有实测数字均可一键重跑。